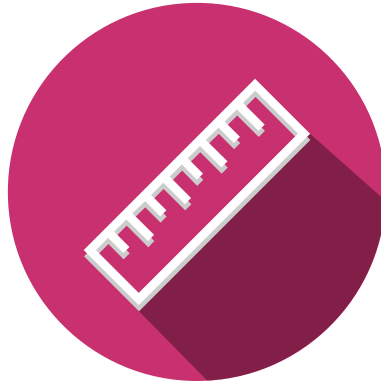




Cursor (Cur)

Vorlesung Digitales Design (DiD)



Orientierung: Systemtechnik (Synd)

Spezialisierung: Infotronics (IT)

Kurs: Digitales Design (DiD)

Autoren: Silvan Zahno, Axel Amand, François Corthay, Christophe Bianchi

Datum: 09.09.2026

Version: v3.1.1



Inhalt

1 Einführung	3
2 Spezifikation	4
2.1 Funktionen	4
2.2 Schaltung	4
2.3 Schaltung	5
2.4 Szenario (Beispiel)	6
2.5 HDL-Designer Projekt	6
3 Komponenten	8
3.1 Schlitten	8
3.2 Motorsteuerungsschaltung	8
3.2.1 Gleichstrommotor	9
3.3 Encoder (Drehgeber)	11
3.4 Reed-Relais	12
3.5 FPGA-Platine	12
3.6 Knöpfe und LEDs	13
4 Bewertung	14
5 Erste Schritte	15
5.1 Tips	15
Glossar	16



1 Einführung

Ziel dieses Projekts ist es, das erworbene Wissen anhand eines praktischen Beispiels am Ende des Semesters konkret anzuwenden. Es umfasst die Steuerung eines Gleichstrommotors, um einen Schlitten präzise entlang einer Gewindespindel zu vordefinierten Positionen zu bewegen. Dieses Positionierungssystem ist in der [Abbildung 1](#) dargestellt.

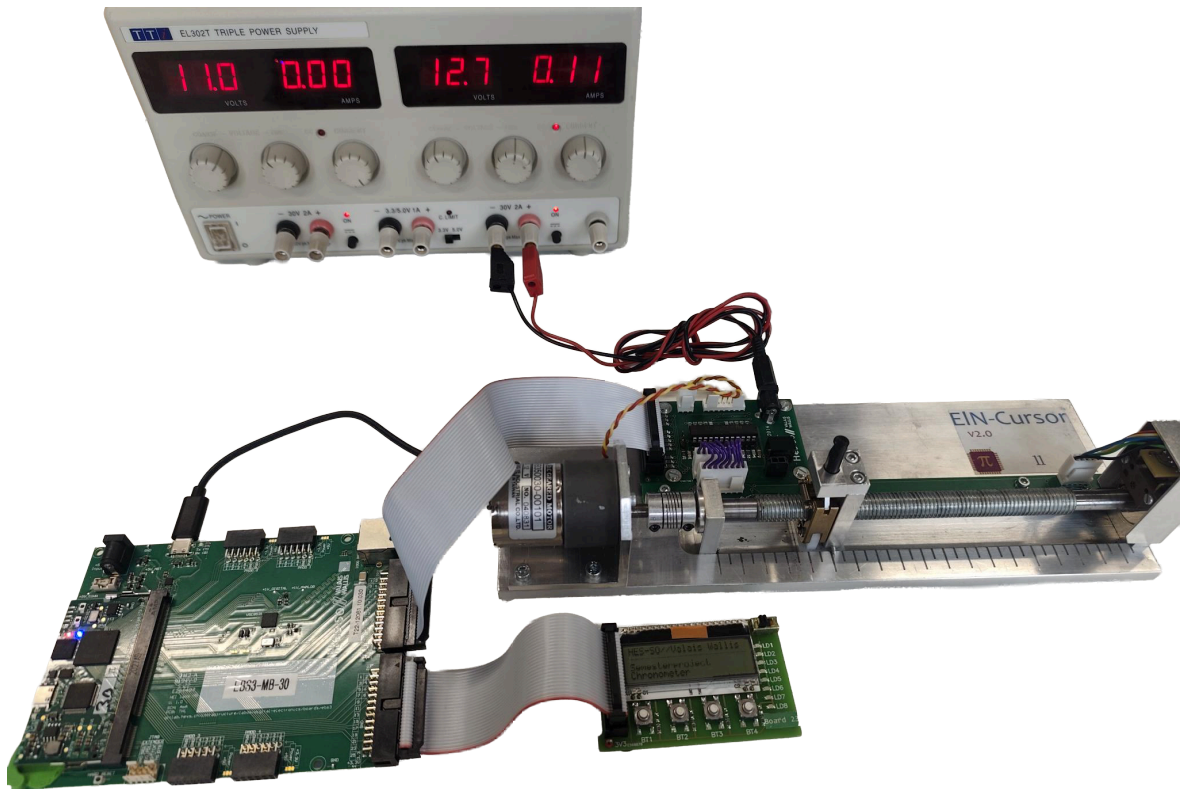


Abbildung 1 - Hardwareaufbau Cursor (EBS3)

Die Mindestspezifikationen (siehe [Abschnitt 2](#)) müssen erfüllt sein, aber die Studierenden werden ermutigt, zusätzliche Funktionen zu implementieren. Beispielsweise kann ein [Liquid Crystal Display \(LCD\)](#)-Bildschirm verwendet werden, um verschiedene Informationen anzuzeigen.



Durch die Implementierung zusätzlicher Funktionen können zusätzliche Punkte in der abschliessenden Bewertung erzielt werden.



2 Spezifikation

2.1 Funktionen

Die Grundfunktionen sind wie folgt definiert:

- Wenn die Taste **restart** gedrückt wird, bewegt sich der Cursor zur Startposition, die durch ein Reed-Relais ([Abschnitt 3.4](#)) in der Nähe des Gleichstrommotors ([Abschnitt 3.2.1](#)) angezeigt wird.
- Wenn die Taste Position_1 gedrückt wird, muss der Cursor zuerst gleichmässig zur Position 1 (p_1) beschleunigen, dann mit voller Geschwindigkeit vorwärts fahren und schliesslich gleichmässig abbremsen, um an Position 1 (p_1) anzuhalten. Dies kann von der Startposition oder von Position 2 aus erfolgen, siehe [Abbildung 2](#).
- Wenn die Taste Position_2 gedrückt wird, muss der Cursor zuerst gleichmässig zur Position 2 (p_2) beschleunigen, dann mit voller Geschwindigkeit vorwärts fahren und schliesslich gleichmässig abbremsen, um an Position 2 (p_2) anzuhalten, siehe [Abbildung 2](#).

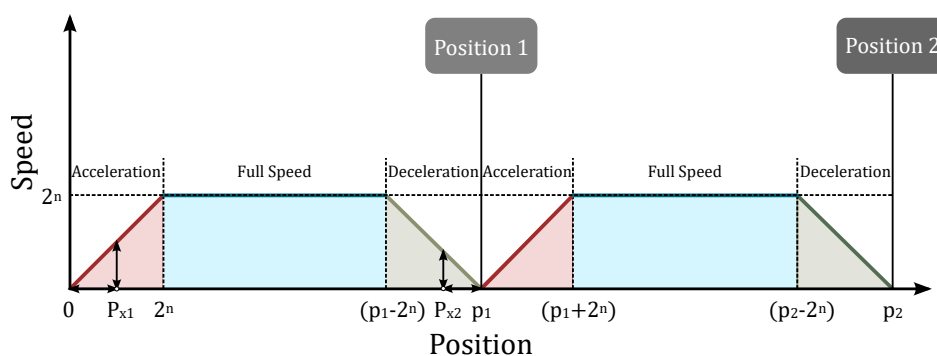


Abbildung 2 - Diagramm der Geschwindigkeit des Schlittens

Die Zielpositionen sind:

- Position 0 (p_0 , Reset-Position) = 3.3 - 3.5 cm je nach mechanischer Montage
- Position 1 (p_1) = 8 cm
- Position 2 (p_2) = 12 cm

2.2 Schaltung

Um die Aufgabe zu erfüllen, wird die folgende Schaltung zum Bewegen des Schlittens bereitgestellt.

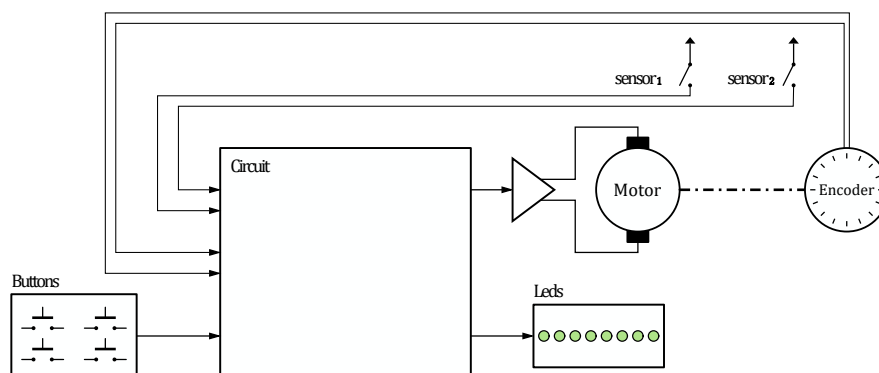


Abbildung 3 - Cursor Schaltung



2.3 Schaltung

Der Schaltungsaufbau ist wie folgt:

- Der Gleichstrommotor ([Abschnitt 3.2.1](#)) wird durch die drei Signale motor_{On} , side_1 , side_2 gesteuert:
 - Der Motor wird aktiviert, wenn das Signal motor_{On} auf 1 gesetzt ist und muss auf 0 zurückgesetzt werden, wenn der Motor nicht laufen soll.
 - Seine Geschwindigkeit wird durch eine **Pulse Width Modulation (PWM)** Modulation gesteuert, die entweder auf das Signal side_1 oder auf das Signal side_2 angewendet wird, je nach gewünschter Drehrichtung.
- Zwei Reed-Relais ([Abschnitt 3.4](#)) sensor_1 und sensor_2 sind an den Enden der Schiene [1] angebracht. Sie erkennen die Anwesenheit des Cursor-Schlittens, indem sie das Vorhandensein eines Magneten mit einem 1 anzeigen.
- Der Encoder ([Abschnitt 3.3](#)) wird verwendet, um die Position des Cursors zu verfolgen bzw. zu zählen. Seine drei Ausgänge, encoder_A , encoder_B und encoder_I , ermöglichen es, die Bewegungen der Spindel zu verfolgen. Die Ausgänge A und B wechseln zwischen 0 und 1 während der Bewegungen mit einer Phasenverschiebung von 90° zwischen ihnen, was es ermöglicht, die Bewegungsrichtung zu bestimmen. Der Ausgang I erzeugt einen 1-Impuls bei jeder vollständigen Umdrehung der Spindel.
- Drei Tasten werden verwendet, um das System zu steuern: restart , go_1 und go_2 . Eine zusätzliche Taste, button_4 , kann für optionale Funktionen verwendet werden. Wenn eine der Tasten gedrückt wird, wechselt das entsprechende Signal auf 1.
- Die Pins testOut können verwendet werden, um zusätzliche Informationen aus dem System auszugeben, z.B. um sie an die **Light Emitting Diodes (LEDs)** für Debugging- oder Visualisierungszwecke anzuschließen.
- Das Signal testMode wird auf 1 gesetzt während der Simulation. Dieses kann verwendet werden, um die Zähler zu verkürzen, die zur Signalerzeugung während der Tests verwendet werden, um diese zu beschleunigen.

Das leere Toplevel Design (**cursor-toplevel-empty.pdf**) zeigt alle Signale, die mit dem **Field Programmable Gate Array (FPGA)** Board verbunden sind [Abbildung 4](#).



Abbildung 4 - Leere Toplevel Schaltung



2.4 Szenario (Beispiel)

In [Abbildung 5](#) werden drei verschiedene Szenarien dargestellt. Zuerst wird die restart-Taste gedrückt und der Schlitten bewegt sich mit voller Geschwindigkeit zur Startposition (sensor₁). Die beiden anderen Szenarien go₂ und go₁ bewegen den Schlitten zur position₂ und zur position₁ respektive, ein variables PWM Signal wird auf die Signale side₂ und side₁ angewendet, um den Schlitten zu beschleunigen und zu verlangsamen.

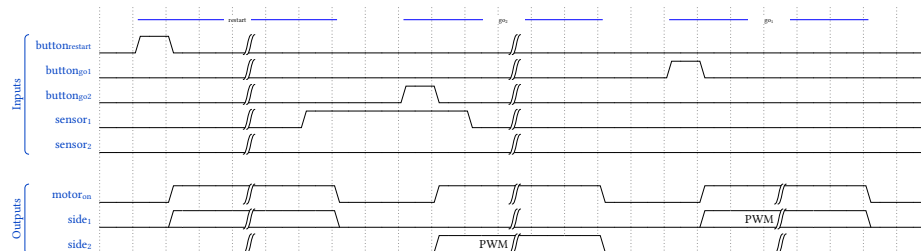


Abbildung 5 - Cursor Szenario



Die oben genannten Szenarien sind Beispiele, die von den Studierenden vervollständigt werden müssen.

2.5 HDL-Designer Projekt

Ein vordefiniertes HDL-Designer Projekt kann von [Cyberlearn](#) oder [Github](#) heruntergeladen oder geklont werden. Die Dateistruktur des Projekts sieht wie folgt aus:

```
did_cursor
+--Board/          # Project and files for programming the FPGA
|   +--concat/     # Complete VHDL file including PIN-UCF file
|   +--ise/        # Xilinx ISE project
+--Cursor/         # Library for the components of the student solution
+--Cursor_test/    # Library for the simulation testbenches
+--doc/            # Folder with additional documents relevant to the project
|   +--Board/      # All schematics of the hardware boards
|   +--Components/ # All data sheets of hardware components
+--img/            # Pictures
+--Libs/           # External libraries which can be used e.g. gates, io, sequential
+--Prefs/          # HDL-Designer settings
+--Scripts/        # HDL-Designer scripts
+--Simulation/     # Modelsim simulation files
```



Der Pfad des Projektordners darf keine Leerzeichen enthalten.



Im Projektordner **doc/** können viele wichtige Informationen gefunden werden. Datenblätter, Projektbewertung sowie Hilfsdokumente für HDL-Designer um nur einige zu nennen.



3 Komponenten

Das System besteht aus 3 verschiedenen Hardwareplatinen, die in der [Abbildung 1](#) zu sehen sind.

- Ein Schlittenaufbau mit einer **Printed Circuit Board (PCB)**-Platine, die den Motor steuert und die Sensoren ausliest ([Abbildung 6](#))
- Eine **FPGA**-Entwicklungsplatine ([Abbildung 14](#))
- Eine Steuerplatine mit 4 Tasten und 8 LEDs ([Abbildung 15](#))

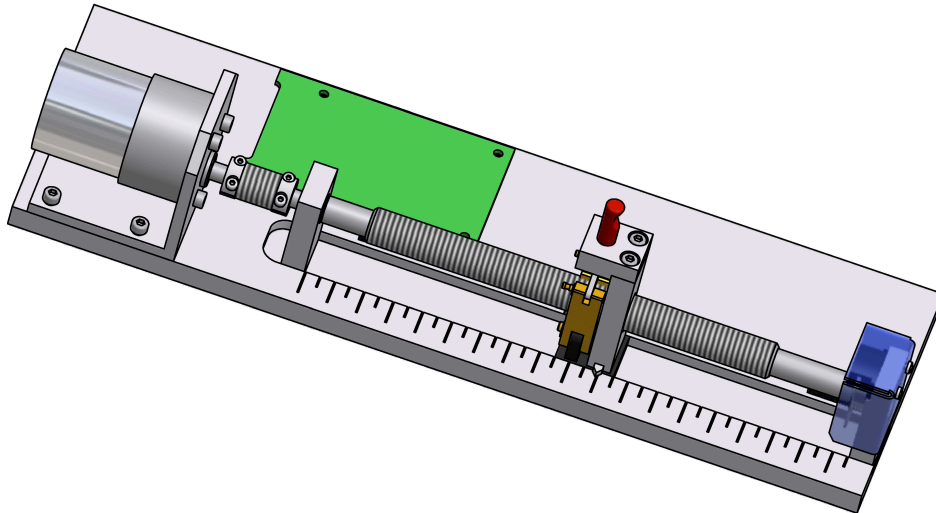


Abbildung 6 - Cursor Schlittenaufbau

3.1 Schlitten

Der Schlittenaufbau beinhaltet den Gleichstrommotor, die beiden Reed-Relais [Abschnitt 3.4](#) sowie den Schlitten und die Spindel. Der Spindelsteigung ist M12x1.75, was bedeutet, dass pro Umdrehung eine Strecke von 1.75mm zurückgelegt wird ([Abbildung 7](#)).

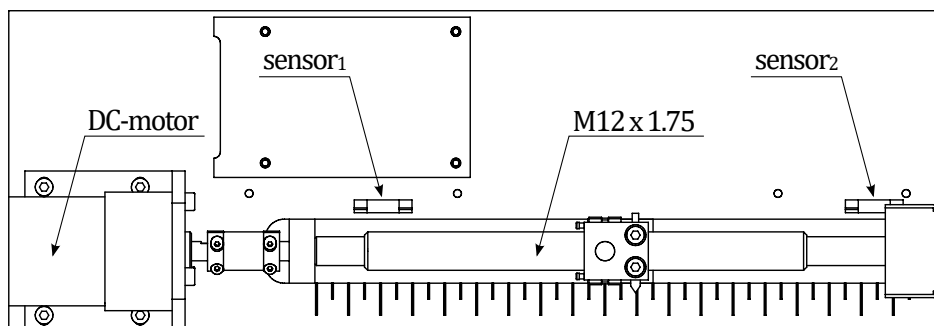


Abbildung 7 - Cursor Schlittenaufbau Detail

3.2 Motorsteuerungsschaltung

Der Gleichstrommotor des Wagens wird mit 12V versorgt. Die Stromversorgungsplatine besitzt eine H-Brücke, die durch digitale Signale gesteuert wird. Auf der Stromversorgungsplatine erzeugt ein 5V-Regler die richtige Spannung für die Versorgung der **FPGA**-Platine [2].



3.2.1 Gleichstrommotor

Der Gleichstrommotor wird von einem L6207 H-Brücken-Treiber [3] gesteuert, siehe Abbildung 8. Die Maximale Schaltfrequenz der H-Brücke beträgt 100kHz. Dies sollte bei der Erstellung des PWM Signales in Betracht gezogen werden.

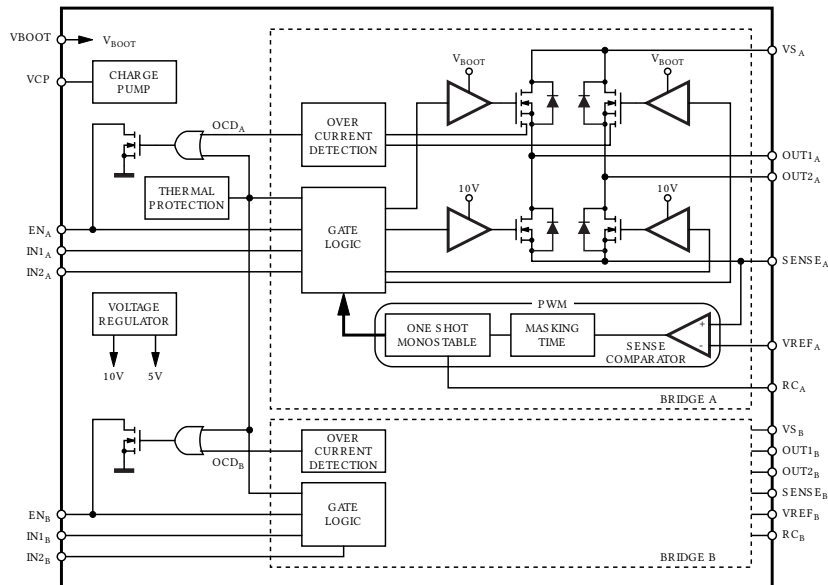


Abbildung 8 - H-Brücke L6207N Schaltung [3]

Um die Geschwindigkeit des Gleichstrommotors anzupassen, muss ein PWM-Signal an die side₁ oder side₂ Signale angelegt werden, wobei die maximale Frequenz 100kHz beträgt. Je länger die Spannung am Motor angelegt wird, desto schneller dreht er sich. Die Richtung des Motors wird gesteuert, indem entweder side₁ oder side₂ mit Spannung versorgt wird.

PWM - Implementierungsstrategie

Eine Strategie besteht darin, einen Sägezahnzähler ($0, 1, 2 \dots 2^N - 2, 2^N - 1, 0, 1, \dots$) zu erstellen und seinen Wert mit einem Schwellenwert zu vergleichen. Die Regel ist, '1' auszugeben, wenn der Zählerwert kleiner als der Schwellenwert ist, und '0' sonst. Durch Ändern des Schwellenwerts wird das Tastverhältnis des PWM-Signals verändert. Auf diese Weise wird die Frequenz des PWM-Signals durch die Geschwindigkeit des Zählers (Sägezahn) definiert, und das Tastverhältnis wird durch den Schwellenwert definiert.

In der Abbildung 9 dreht sich der Motor langsamer mit dem grünen Signal als mit dem blauen Signal und als mit dem roten Signal.

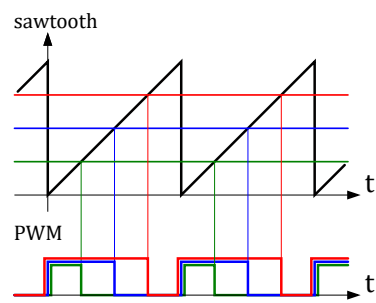


Abbildung 9 - PWM Signale



Je höher die Anzahl der Schritte (Bits) der **PWM**, desto präziser ist die Geschwindigkeitsregelung. Allerdings können zu viele Schritte aufgrund von Steuerungssystemen und Motoreigenschaften zu keinem bemerkenswerten Geschwindigkeitsunterschied führen. Eine gute Basis ist die Verwendung von 8 Bits für die **PWM**-Auflösung, was 256 Geschwindigkeitsstufen von 0 bis 255 ergibt.

Rampen - Implementierungsstrategien

Um Beschleunigungs- und Verzögerungsrampen zu erstellen, ist eine gängige Methode die Verwendung eines trapezförmigen Geschwindigkeitsprofils. Dieses Profil besteht aus drei Phasen: Beschleunigung, konstante Geschwindigkeit und Verzögerung, wie in **Abbildung 10** dargestellt.

Mehrere Strategien existieren, um solche Rampen zu erstellen. Typischerweise basieren sie auf der Zeit. Aber ein solches System erfordert eine Rückkopplungsschleife, solange eine Positionsregelungsschleife.

Eine einfachere, akzeptable Lösung besteht darin, die Rampen auf der Position basierend auf $v_x = m * x + b$ zu erstellen.

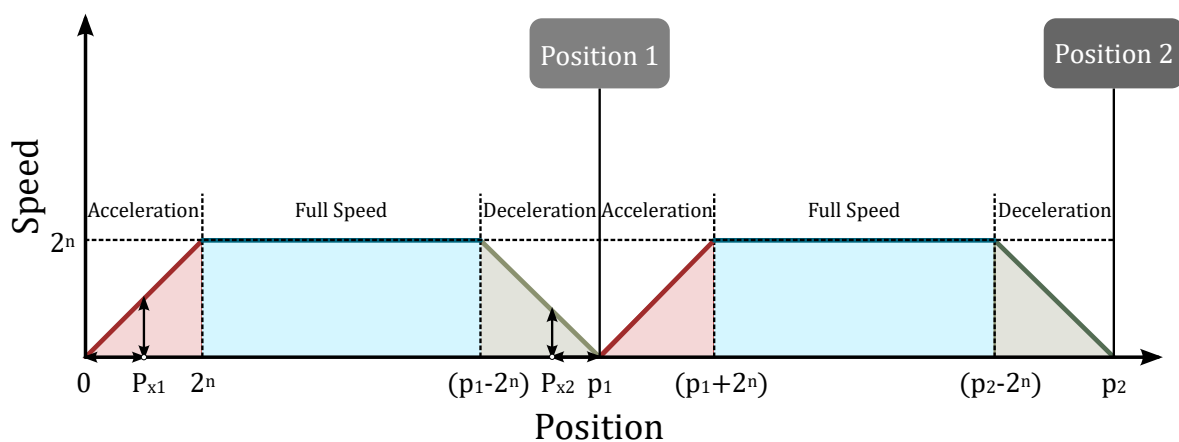


Abbildung 10 - Diagramm der Geschwindigkeit des Schlittens

3.3 Encoder (Drehgeber)

Der Winkel der Spindel kann mithilfe eines **inkrementalen Drehgebers** gemessen werden. Das Modell, das im Aufbau verwendet wird, ist ein AEDB-9140-A12 [4] (**Abbildung 11**) mit 2 Rückführungskanälen mit 500 **Counts per Revolution (CPR)** (Impulse pro Umdrehung) pro Kanal, dargestellt in der **Abbildung 12**.

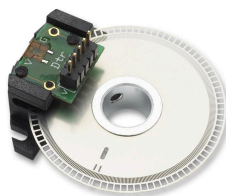


Abbildung 11 - Encoder AEDB-9140-A12 [4]

Encoder - Implementierungsstrategie

Es ist möglich, ihn auf zwei Arten zu verwenden:

- Durch die Verwendung eines einzigen Kanals, was eine Auflösung von $500 \frac{\text{Impulse}}{\text{Umdrehung}}$ ergibt, d.h. $1'000 \frac{\text{Flanken}}{\text{Umdrehung}}$, wenn sowohl steigende als auch fallende Flanken gezählt werden.



- Durch die Verwendung beider Kanäle als **QEI (Quadrature Encoder Interface)**, indem die steigenden und fallenden Flanken beider Kanäle gezählt werden, wodurch die Auflösung auf $2000 \frac{\text{Flanken}}{\text{Umdrehung}}$ erhöht wird. Diese Schnittstelle ermöglicht auch die Erkennung der Drehrichtung des Motors und ist eine gängige Methode zur Steuerung von Gleichstrommotoren mit Rückführungs-Encodern in der Industrie.

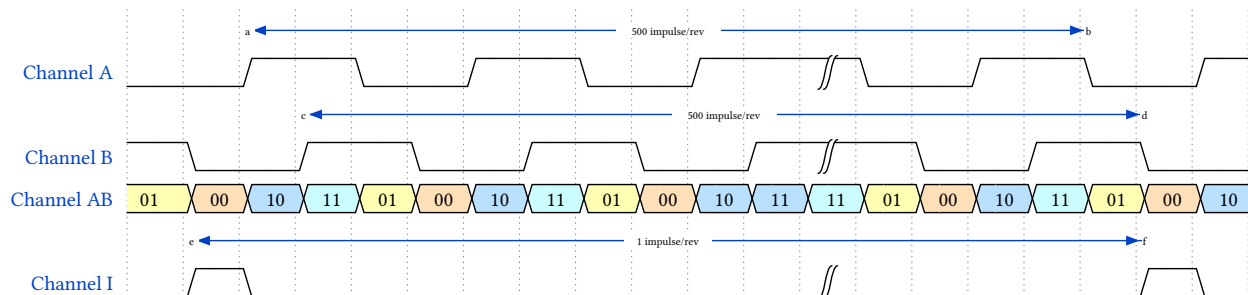


Abbildung 12 - Signale inkrementaler Drehgeber

3.4 Reed-Relais

Das Reed-Relais ist ein Schalter, der mit Hilfe von Elektromagneten umgeschaltet werden kann [1]. Wenn sich ein Magnet in der Nähe des Sensors befindet, schliesst sich der Kontakt (Abbildung 13). 2 Reed-Relais werden verwendet (sensor_1 und sensor_2), um die linke und rechte Grenze des Schlittens zu identifizieren.

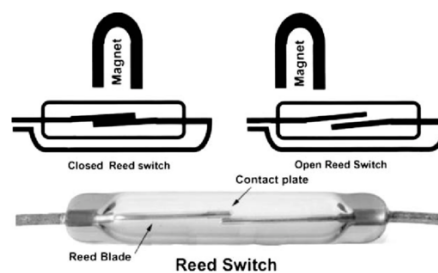
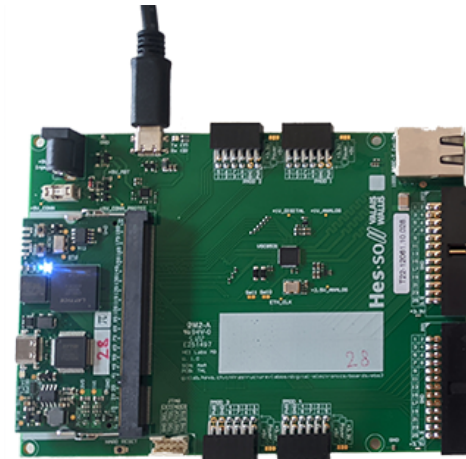


Abbildung 13 - Reed Schalter [5]

Die Signale sensor_1 und sensor_2 sind ,1', wenn der Kontakt geschlossen ist (Magnet in der Nähe), sonst ,0'.

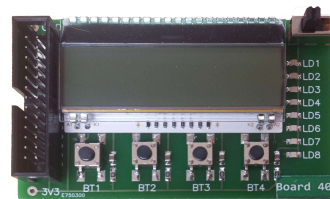
3.5 FPGA-Platine

Die Hauptplatine ist die FPGA-EBS3 Laborentwicklungsplatine der Schule [6]. Sie beherbergt einen **FPGA** Chip **Lattice LFE5U-25F** und verfügt über viele verschiedene Schnittstellen (**Universal Asynchronous Receiver Transmitter (UART)**, **Universal Serial Bus (USB)**, Ethernet, etc.). Der benutzte Oszillator erstellt ein Taktsignal (**clock**) mit einer Frequenz von $f_{\text{clk}} = 100\text{MHz}$, reduziert durch PLL auf $f_{\text{clk}} = 60\text{MHz}$.

Abbildung 14 - EBS3 **FPGA** Platine [6]

3.6 Knöpfe und LEDs

Die Platine mit den Knöpfen und **LEDs** [7] wird an die **FPGA** Platine angeschlossen. Sie hat 4 Tasten und 8 **LEDs**, die im Design verwendet werden können. Falls gewünscht kann diese Platine mit einer **LCD** Anzeige ausgestattet werden [8], [9].

Abbildung 15 - Knöpfe-**LEDs**-**LCD** Platine [7]



4 | Bewertung

Im Ordner **doc/** zeigt die Datei **evaluation-bewertung-cursor.pdf** das detaillierte Bewertungsschema, [Tabelle 1](#).

Die Schlussnote beinhaltet den Bericht, den Code sowie eine Präsentation eurerseits des Systems.

Evaluierte Aspekte	Punkte
Bericht	55
Einleitung	3
Spezifikation	5
Entwurf	20
Verifizierung und Validation	10
Integration	9
Schlussfolgerung	3
Formale Aspekte des Berichtes	5
Funktionalität der Schaltung	30
Qualität der Lösung	10
Präsentation	10
Total	105

Tabelle 1 - Bewertungsraster



Das Bewertungsraster gibt bereits Hinweise über die Struktur des Berichtes. Für einen guten Bericht konsultieren Sie das Dokument „Wie verfasst man einen Projektbericht?“ [\[10\]](#).



5 | Erste Schritte

Um mit dem Projekt zu beginnen, kann folgendermassen vorgehen werden:

- Lest die obigen Spezifikationen und Informationen genau durch.
- Schaut euch die Hardware und testet das vorprogrammierte Programm.
- Stöbert durch die Dokumente im Ordner **doc/** eures Projektes.
- Entwickelt ein detailliertes Blockdiagramm. Die Signale und deren Funktionen solltet Ihr erklären können.
- Implementierung und Simulation der verschiedenen Blöcken.
- Testen der Lösung auf der Platine und finden etwaiger Fehler 🐛.

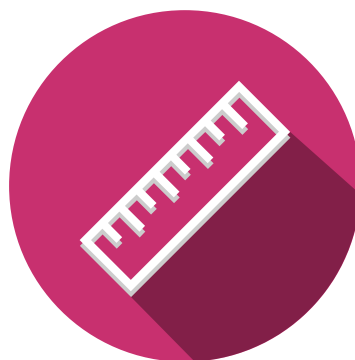
5.1 Tips

Anbei noch einige zusätzlichen Tips um Probleme und Zeitverlust zu vermeiden:

- Teilt das Problem in verschiedene Blöcke auf, benutzt hierzu das leere Toplevel Dokument (**cursor-toplevel-empty.pdf**). Es ist ein ausgeglichener Mix zwischen Anzahl Komponenten und Komponentengrösse empfohlen.
- Analysiert die verschiedenen Ein- sowie Ausgangssignale, hierzu sollten teilweise die Datenblätter zu Hilfe genommen werden.
- Beachtet bei der Erstellung des Systems das DiD Kapitel „Methodologie für die Entwicklung von digitalen Schaltungen (MET)“ [\[11\]](#).
- Es wird empfohlen das System in zwei Schritten zu realisieren.



Vergesst nicht Spass zu haben.





Glossar

CPR – Counts per Revolution [11](#)

FPGA – Field Programmable Gate Array [5](#), [8](#), [12](#), [13](#)

LCD – Liquid Crystal Display [3](#), [13](#)

LED – Light Emitting Diode [5](#), [8](#), [13](#)

PCB – Printed Circuit Board [8](#)

PWM – Pulse Width Modulation [5](#), [6](#), [9](#), [10](#), [11](#)

UART – Universal Asynchronous Receiver Transmitter [12](#)

USB – Universal Serial Bus [12](#)



Literatur

- [1] „Reed Relay“. 5. Dezember 2020. Zugriffen: 24. November 2021. [Online]. Verfügbar unter: https://en.wikipedia.org/w/index.php?title=Reed_relay&oldid=992433034
- [2] Olivier Walpen, „Schematic: Cursor Chariot Power Circuit“. 2009.
- [3] STMicroelectronics, „Datasheet: DMOS Dual Full Bridge Driver with PWM Current Controller“. 2003.
- [4] Agilent Technologies, „Datasheet Agilent AEDB-9140 Series Three Channel Optical Incremental Encoder Modules with Codewheel, 100 CPR to 500 CPR“. 2005.
- [5] „Magnetic-Reed-Switch-Above-Closed-and-open-reed-switch-in-response-to-magnet-placement.Png (850×345)“. Zugriffen: 24. November 2021. [Online]. Verfügbar unter: <https://www.researchgate.net/profile/Sidakpal-Panaich-2/publication/51169357/figure/fig1/AS:394204346896388@1470997048549/Magnetic-reed-switch-Above-Closed-and-open-reed-switch-in-response-to-magnet-placement.png>
- [6] A. Amand und S. Zahno, „FPGA-EBS3 Electornic Technical Documentation“. 2022.
- [7] Silvan Zahno, „Schematic: Parallelport HEB LCD V2“. 2014.
- [8] Sitronix, „Datasheet Sitronix ST7565R 65x1232 Dot Matrix LCD Controller/Driver“. 2006.
- [9] Electronic Assembly, „Datasheet: DOGM Graphics Series 132x32 Dots“. 2005.
- [10] Christophe Bianchi, François Corthay, und Silvan Zahno, „Wie Verfasst Man Einen Projektbericht?“. 2021.
- [11] François Corthay, Silvan Zahno, und Christophe Bianchi, „Méthodologie de Conception de Cicuits Numériques“. 2021.